
AN0027-EC NB-IoT Secure Boot Introduction

_V1.1

EigenCOMM Wireless Microcontroller

Document Description

This document describes the secure boot feature of EC616 NB-IOT Chip

Contents

1.	General Description.....	3
2.	Secure boot components.....	3
3.	Boot Process.....	5
3.1	Secure Boot Steps.....	5
3.2	Boot Flows.....	6
3.2.1	BootROM flow	6
3.2.2	Bootloader Flow	8
4.	Building and Booting a secure system	9
4.1	Generate public/private key pairs	9
4.2	Config secure option in SDK.....	9
4.2.1	Config the config_secure.ini.....	9
4.2.2	Config the Makefile for bootloader	10
4.2.3	Config the Makefile for system	10
4.2.4	Enable secure compile	11
4.3	Build image heads and images	11
4.3.1	Geneate bootload image	11
4.3.2	Geneate system image	11
4.4	Program efuse bits	11
4.5	Program bootloader and system image	11
5.	Hash and ECDSA algorithm.....	12
6.	References	12
6.1	Reference Document	12
6.2	Terms and Interpretation.....	13
6.3	Safety and Warning.....	14
7.	Version.....	15
8.	About US.....	16

1. General Description

Secure boot is a feature for ensuring the chip only run your code, data loaded from flash is verified on each reset.

Secure boot of EC616 chip uses Elliptic Curve Digital Signature Algorithm (ECDSA) and SHA-256 as the hashing algorithm. The ECDSA algorithm generates asymmetric key (a public key and a related private key) used to sign and verify the image, and a digital signature scheme can provide to protect the data integrity of the image (the assurance that data has not been altered by unauthorized or unknown means).

Bootloading a device securely starts with BootROM code loading the bootloader image, and continues serially with the bootloader loading the system image. With a secure boot foundation established by the BootROM code, the chain of trust is created by the successive authentication of the image loaded into the device.

The public key used in BootROM to verify the bootloader image is stored in efuse, and a secure enable bit in the efuse is used to enable the secure boot flow. Once the efuse bits are burned, it can not be changed, and permanently binds with the chip. The public key used in the bootloader to verify the system image is stored in the trusted bootloader image.

2. Secure boot components

1. eFUSE

The Efuse array is on-chip one-time programmable (OTP) NVM, the Efuse array stores the 512-bit public key. It is also used to control security functions, including secure boot enable/disable, debug enable/disable, download enable/disable or the JTAG port enable/disable.

HW -PLAT field efuse bits

Name	Usage	Default value	size	Control	BYTE ADDR
JTAG flag	0- enable JTAG 1- disable JTAG	0	1 bit	SOC reg	[18]
Secure boot flag	0- secure boot disable 1- secure boot enable	0	1 bit	Software Interface	[18]
Debug flag	0- debug enable 1- debug disable	0	1 bit	Software Interface	[18]
Download flag	0- download enable 1- download disable	0	1 bit	Software Interface	[18]
Flash Sw Reset Enable	0- flash not reset when boot	0	1bits	Software Interface	[18]

	1- flash sw reset when boot				
Reserved bits	spare bits for User	0	3bits	Software Interface	[18]

SW -PLAT field efuse bits

Name	Usage	Default value	size	Control	BYTE ADDR
SW RSVD BYTES	spare bits for User	0	32bits	Software Interface	[63: 60]
ECDSA public key	Public key	0	512 bits	Software Interface	[127:64]

2. BootROM

The BootROM is 64k mask programmed boot Read Only Memory which contains BootROM code. The BootROM is not visible to the user or writable. The BootROM code initializes essential clocks , reads efuse info at reset or power on and loads the bootloader image.

The BootROM code will run integrity checks with the image id and the hash value in the bootloader image head if needed. When secure boot is enabled in efuse bits, the BootROM will execute ECDSA verification code to verify the signature in the bootloader image head with public key which readed from efuse bits.

The BootROM will then transfer to the bootloader entry address.

3. ECSecTools

ECSecTools is a PC base software with ECDSA and HASH algorithm, provides support of security functions. ECSecTools generates ECDSA public/private key pairs. ECSecTools uses ECDSA alogrithom generate signatures of the image with private key, uses SHA256 to generate hashes, output a image head binary file containing hash and signature for secure verifying.

4. FlashToolCLI

FlashToolCLI is a PC base software to program image binary to FLASH.

5. SDK

SDK is software development kits containing toolchains, drivers , middleware, protocols, bootloader projects and system projects. SDK also supports to integrate the secure boot feature and generate secure boot image files and image head files.

6. Image Head

The Image Head defines characteristics of images. The image ID and Head hash values in the image head allows the secure code to run integrity checks. When secure boot is enabled, the secure code will verify the image using ECDSA signature in the image head.

7. Image

An image is a binary file which contains the bitstream and software. Typically, a bootloader image is named “bootloader.bin” and a system image is “app-demo-flash.bin”.

8. Bootloader

The bootloader code is programmed into on-chip XIP flash memory, and executed in xip mode. The bootloader initializes essential clocks , drivers, reads efuse info, and load system image.

The bootloader code will run integrity checks with the image id and the hash value in the system image head if needed. When secure boot is enabled in efuse bits, the bootloader will execute ECDSA verification code to verify the signature in the system image head with public key which integrated into bootloader code.

The bootloader will then transfer to the system entry address.

9. System

The system code is programmed into on-chip XIP flash memory, and executed in xip mode. The system is the main image containing all the drivers, protocols, firmwares and applications.

10. On-Chip XIP Flash

EC616 chip has a on-chip serial flash memory to store the code and data. The chip excute code by accessing the serial flash memory through a hardware controller with XIP technology. XIP (execute-in-place) technology enables a flash memory incorporated within the soc chip to excute code stored in the flash memory without additional memory copy.

3. Boot Process

This section provides an overview of boot steps, boot flows. Following this, the software used in boot, including BootROM code, bootloader is discussed.

3.1 Secure Boot Steps

Below show steps to develop a secure embedded system.

1. Generate ECDSA public/private keys
2. Program ECDSA public key for bootloader and secure enable to efuse array.
3. Use SDK and tools to generate a 'VerifyPubkey.c' with the ECDSA public key for system.
4. Use SDK and tools to create a signed bootloader image head(`bootloader_head.bin`), and bootloader image (`bootloader.bin`) include code of 'VerifyPubkey.c'.
5. Use SDK and tools to create a signed system image head(`system_head.bin`), and system image(`app-demo-flash.bin`).
6. Use FlashToolCLI.exe to program the `bootloader_head.bin`, `bootloader.bin`, the `system_head.bin` and `system.bin` to Flash.

3.2 Boot Flows

To boot the ec616 devices, software (bootloader and system) is developed on a PC using SDK. And the image is created by SDK and tools.

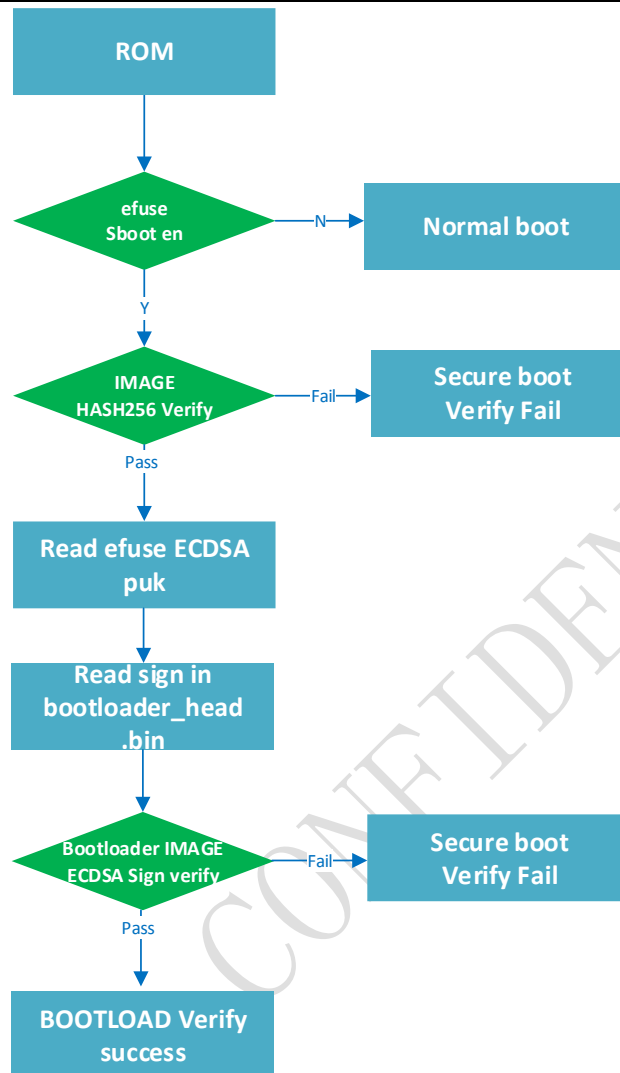
3.2.1 BootROM flow

Figure below shows the The BootROM code flow.

The BootROM code reads efuse values to determine wheter the boot is non-secure or secure, and if secure, verifies the integrity of the bootloader image by computing a hash and compare the result with the hash value stored in bootloader image head. If integrity check fails, the boot will be aborted.

The BootROM code reads ECDSA public key from efuse, hash value and signature, uses the ECDSA algorithm to verify the digital signature correctness. If the verify fails, the boot is aborted.

If all secure checks pass, the BootROM will transfer to bootloader code.



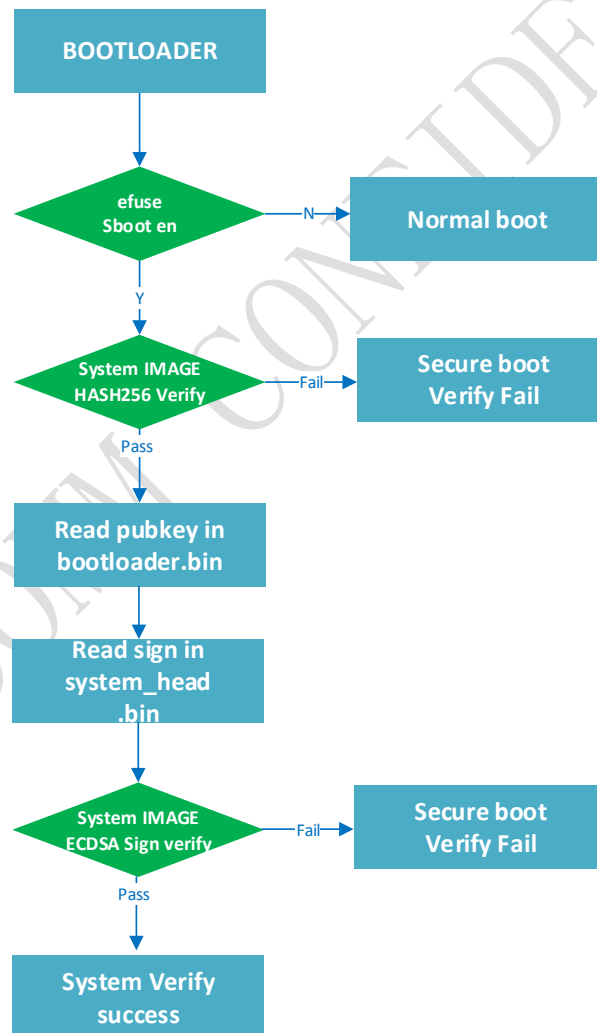
3.2.2 Bootloader Flow

Figure below shows the The bootloader code flow.

The bootloader code reads efuse values to determine wheter the boot is non-secure or secure, and if secure, verifies the integrity of the system image by computing a hash and compare the result with the hash value stored in system image head. If integrity check fails, the boot will be aborted.

The bootloader will execute ECDSA verification code to verify the signature in the system image head with public key which integrated into bootloader code. If the verify fails, the boot is aborted.

If all secure checks pass, the bootloader will transfer to system code.



4. Building and Booting a secure system

This section provides the steps to develop a secure boot system using the SDK and tools.

The required tasks for a secure boot are:

1. Generate ECDSA public/private keys pairs for bootloader and system.
2. Config the secure option in SDK.
3. Use SDK to generate secure image heads and images.
4. Use tools to program the public keys for bootloader to efuse array.
5. Use tools to program the image and image head to flash.

4.1 Generate public/private key pairs

Use below cmd to generate ECDSA key pairs file for signing and verifying bootloader.

```
ECSecTools.exe genkey .\case_secure\securitybl_priv.pem .\case_secure\securitybl_pub.pem
```

Use below cmd to generate ECDSA key pairs file for signing and verifying system.

```
ECSecTools.exe genkey .\case_secure\securitysys_priv.pem .\case_secure\securitysys_pub.pem
```

4.2 Config secure option in SDK

4.2.1 Config the config_secure.ini

Specify the config file 'config_secure.ini' for ECSecTools in Makefile, and the file content is like below:

```

config_secure.ini x
1  [config]
2  line_0_com = COM29
3  agbaud=921600
4
5  [security]
6  hashtype = 1
7  privkeyfilebl = .\case_secure\securitybl_priv.pem
8  pubkeyfilebl = .\case_secure\securitybl_pub.pem
9  privkeyfile = .\case_secure\securitysys_priv.pem
10 pubkeyfile = .\case_secure\securitysys_pub.pem
11
12 ;agentboot.bin file infomation
13 [agentboot]
14 agpath = .\agentboot.bin
15 agloadskip = 0
16
17 ;bootloader.bin file infomation
18 [bootloader]
19 headpath = ..\..\out\ec616_0h00\bootloader\bootloader_head.bin
20 blpath = ..\..\out\ec616_0h00\bootloader\bootloader.bin
21 blloadskip = 0
22
23 ;system.bin file infomation
24 [system]
25 headpath = ..\..\out\ec616_0h00\at_command\app-demo-flash_head.bin
26 syspath = ..\..\out\ec616_0h00\at_command\app-demo-flash.bin
27 sysloadskip = 0
28
29 ;control such as reset before download
30 [control]
31 reset = 0
32

```

The privkeyfilebl/pubkeyfilebl var in bootloader security section should be set to the path of securitybl-priv.pem/securitybl-pub.pem, and the privkeyfile/pubkeyfile var should be set to the path of securitysys-priv.pem/securitysys-pub.pem.

4.2.2 Config the Makefile for bootloader

Edit the MakeProj.rules file under bootloader projects, and config the EC_SECURE_TOOL_CFG var like this.

```
EC_SECURE_TOOL_CFG ?= $(TOP)/PLAT/tools/FlashRun/config_secure.ini
```

4.2.3 Config the Makefile for system

Edit the MakeProj.rules file under system projects, and config the EC_SECURE_TOOL_CFG var like this.

```
EC_SECURE_TOOL_CFG ?= $(TOP)/PLAT/tools/FlashRun/config_secure.ini
```

4.2.4 Enable secure compile

Edit the Makefile under plat\project\ec616_0h00\apps\bootload\armc, and set CONFIG_PROJ_APP_SECURITY_BOOT var like this.

```
CONFIG_PROJ_APP_SECURITY_BOOT = y
include $(TOP)/PLAT/project/$(TARGET)/apps/$(PROJECT)/code/Makefile.inc
include $(TOP)/PLAT/project/$(TARGET)/apps/$(PROJECT)/armcc/MakeProj.rules
```

Edit the Makefile under plat\project\ec616_0h00\apps\at_command\armcc, and set CONFIG_PROJ_APP_SECURITY_BOOT var like this.

```
CONFIG_PROJ_APP_SECURITY_BOOT = y
include $(TOP)/PLAT/project/$(TARGET)/apps/$(PROJECT)/armcc/MakeProj.rules
include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

4.3 Build image heads and images

4.3.1 Geneate bootload image

Run keilbuilt.bat bootloader , the build process will generate 'VerifyPubkey.c', which finally compiled into bootloader.bin, also with the secure config a bootloader_head.bin with hash digest and ECDSA sign will be generated.

4.3.2 Geneate system image

Run keilbuilt.bat at_command , the build process will output app_demo_flash.bin, also with the secure config a system_head.bin with hash digest and ECDSA sign will be generated.

4.4 Program efuse bits

Use the ECSecTools to program the the private key getted from securitybl-pub.pem the secure enable bit to efuse.

4.5 Program bootloader and system image.

Config FlashToolCli.exe with config_secure.ini, run below command and burn the bootloader and image to flash.

```
FlashToolCli.exe -cfgfile config_secure.ini burn
```

5. Hash and ECDSA algorithm

The SHA-256 and ECDSA(curve secp256k1) algorithm is used for secure boot, ECDSA algorithm is described by [RFC6979](#).

A private/public key pair used in ECDSA authentication have significant security advantages over cryptography which only uses private keys. The private key is used at the manufacturing facility which usually has physical security. The public key is loaded into embedded device. If an adversary steals the embedded device and extracts the public key, the damage is limited.

6. References

6.1 Reference Document

Table 6-1. Reference Document

NO.	Document Name	Comments
【1】		
【2】		

6.2 Terms and Interpretation

Table 6-2. Terms and Interpretation

Terms	Interpretation

6.3 Safety and Warning

EiGENCOMM CONFIDENTIAL

7. Version

Version	Date	Comments
Draft	2018-11-11	
A	2018-12-01	

8. About US

Shanghai EigenCOMM Technology Co.,Ltd is established in Feb 2017 in Zhangjiang Hi-Tech park, Shanghai China (www.eigencomm.com) . EigenCOMM focuses on cellular based IoT chipset and solution. With accumulation of experience over the years in algorithm, L1/L2/L3 protocol, SoC, RF , platform & application software as well as the communication architecture and system, team makes its way to IoT industry.

The ultra-low power NB-IoT chipset EC616(S) has already enter mass production stage. The 4G Cat.1 chipset EC618 is in engineering samples stage, 5G is also in the R&D stage.

Eigencomm Technologies Ltd.

Address: Room 707, No 298 Xiangke Road, Shanghai, China

Zip Code: 201210

Phone:

E-mail:

Website: <http://www.eigencomm.com>

Support Window

E-Mail: support@eigencomm.com